

TR? L?I TO?N B? C?U H?I V?N ??P - STAYSAGA

M?n: C?c c?ng ngh? m?i trong ph?t tri?n ph?n m?m. T?i li?u g?m 57 c?u, m?i c?u c? 2 h??ng tr? l?i: ??p ?n chu?n theo t?i li?u/ngu?n ch?nh th?c v? ??p ?n ?p d?ng tr?c ti?p v?o app StaySaga.

Ngu?n n?i b? d?ng ?? ?p d?ng StaySaga: README.md, supabase/migrations, docs/qa-full-test-report.md, docs/AI_PROMPTS.md v? m? ngu?n trong src/.

C?u 1. Sự khác biệt giữa Client Component và Server Component trong Next.js là gì? Khi nào nên dùng loại nào?

1) ??p ?n chu?n: Server Component ch?y v? render ? server, ph? h?p ph?n ??c d? li?u, SEO, n?i dung ?t t??ng t?c v? gi?m JavaScript g?i v? tr?nh duy?t. Client Component ch?y ? browser, c?n khi d?ng useState/useEffect, event handler, browser API ho?c realtime UI. N?n ?? m?c ??nh l? Server Component, ch? t?ch ph?n th?t s? t??ng t?c sang Client Component.

2) Theo app StaySaga: Trong StaySaga, trang ??c d? li?u nh? /homestays, /bookings, /admin, /host ?u ti?n Server Component ?? l?y Supabase server-side. C?c ph?n t??ng t?c nh? FavoriteButton, form checkout, wizard host, toast v? RealtimeSubscription l? Client Component.

C?u 2. App Router trong Next.js 13+ khác với Pages Router ở điểm nào? Nêu ít nhất 3 điểm khác biệt chính.

1) ??p ?n chu?n: App Router d?ng th? m?c app/ thay v? pages/, c? layout l?ng nhau, loading/error theo route segment, Server Components m?c ??nh, Route Handlers v? Server Actions. Pages Router ch? y?u d?a v?o pages/, getServerSideProps/getStaticProps v? API routes ki?u c?.

2) Theo app StaySaga: StaySaga d?ng App Router trong src/app v?i c?c route /homestays, /bookings, /host, /admin v? /api. D? ?n t?n d?ng layout/page theo th? m?c, Server Components m?c ??nh, Server Actions trong src/core/*/actions.ts v? Route Handler cho API/webhook.

C?u 3. Server Actions trong Next.js là gì? Cho biết cú pháp khai báo một Server Action.

1) ??p ?n chu?n: Server Action l? h?m async ch?y tr?n server ?? x? l? mutation nh? submit form, ghi DB, g?i revalidatePath/redirect. Khai b?o b?ng directive "use server" ? ??u file ho?c trong th?n h?m async, r?i truy?n h?m ?? v?o form action ho?c g?i t? Client Component theo c? ch? c?a Next.js.

2) Theo app StaySaga: StaySaga c? nhi?u file "use server" nh? src/core/bookings/actions.ts, src/core/auth/actions.ts, src/core/favorites/actions.ts. V? d? action ??t ph?ng ch?y tr?n server ?? ki?m tra session, t?nh l?i t?ng ti?n, ch?ng double booking r?i revalidate ???ng d?n li?n quan.

C?u 4. Các phương thức Data Fetching nào được hỗ trợ trong Next.js? Khi nào dùng fetch() trực tiếp, khi nào dùng React Query / SWR?

1) ??p ?n chu?n: Next.js h? tr? fetch trong Server Component/Route Handler v?i cache v? revalidation, rendering ??ng ho?c t?nh t?y d? li?u. D?ng fetch tr?c ti?p cho d? li?u server-side v? SEO; d?ng SWR/React Query khi c?n fetching ph?a client, refetch theo focus, polling, optimistic update ho?c tr?ng th?i cache ph?c t?p tr?n UI.

2) Theo app StaySaga: StaySaga ch? y?u fetch d? li?u b?ng Supabase server client trong Server Component/Server Action. N?u c?n UI realtime ho?c tr?ng th?i client nh? b? l?c, toast, wizard, c? th? d?ng state/client fetching; hi?n d? ?n ch?a c?n React Query v? Supabase + App Router ?? ? cho lu?ng ch?nh.

C?u 5. Trong Next.js, file layout.tsx có vai trò gì và khác gì so với page.tsx?

1) ??p ?n chu?n: layout.tsx l? khung d?ng chung cho route v? c?c route con, th??ng ch?a html/body, provider, navbar/sidebar v? c? th? gi? state khi chuy?n trang con. page.tsx l? n?i dung ch?nh c?a ??ng m?t route c? th?.

2) Theo app StaySaga: layout.tsx c?a StaySaga b?c c?u tr?c chung, provider v? shell theo nh?m route; page.tsx l? n?i dung t?ng m?n nh? danh s?ch homestay, chi ti?t booking ho?c dashboard admin. Nh? layout, navbar/shell kh?ng ph?i vi?t l?i cho t?ng page.

C?u 6. Supabase Auth hỗ trợ những phương thức xác thực nào? Liệt kê ít nhất 4 phương thức.

1) ??p ?n chu?n: Supabase Auth h? tr? email/password, magic link/OTP, OAuth nh? Google/GitHub/Facebook, phone SMS, SSO/SAML v? anonymous sign-in t?y c?u h?nh. ?ng d?ng ch?n ph??ng th?c d?a tr?n y?u c?u b?o m?t v? tr?i nghi?m ??ng nh?p.

2) Theo app StaySaga: StaySaga d?ng Supabase Auth cho ??ng k?, ??ng nh?p, ??ng xu?t, callback OAuth v? qu?n l? session qua @supabase/ssr. README n?u email/password, Google/Facebook OAuth v? qu?n/??t l?i m?t kh?u l? c?c lu?ng x?c th?c ch?nh.

Câu 7. Row Level Security (RLS) trong Supabase là gì? Mục đích của RLS?

1) ??? ?n chu:n: RLS là c? ch? ph?n quy?n ? c?p t?ng d?ng trong PostgreSQL. Khi b?t RLS, m?i SELECT/INSERT/UPDATE/DELETE ph?i th?a policy, gi?p ng?n ng??i d?ng ??c ho?c s?a d? li?u kh?ng thu?c quy?n c?a m?nh ngay c? khi client g?i th?ng database API.

2) Theo app StaySaga: StaySaga b?t RLS cho profiles, homestays, bookings, favorites, reviews, notifications. Policy gi?p kh?ch ch? qu?n l? booking/favorite/review c?a m?nh, host qu?n l? homestay c?a m?nh v? admin c? quy?n r?ng h?n qua role.

Câu 8. Các thành phần chính của Supabase được nhắc đến trong môn học là gì? Liệt kê đầy đủ.

1) ??? ?n chu:n: C?c th?nh ph?n ch?nh c?a Supabase g?m PostgreSQL Database, Auth, Storage, Realtime, Edge Functions, API t? sinh, RLS/policies v? c? th? c? Vector cho t?m ki?m ng? ngh?a.

2) Theo app StaySaga: Trong StaySaga, Supabase d?ng cho Auth, PostgreSQL, RLS, Storage bucket homestay-images v? Realtime. Database ch?nh g?m profiles, homestays, homestay_images, amenities, bookings, favorites, reviews, notifications.

Câu 9. Cấu trúc một Row Level Security Policy cơ bản trong Supabase gồm những thành phần nào?

1) ??? ?n chu:n: M?t RLS policy th?ng g?m t?n policy, b?ng, thao t?c ?p d?ng nh? SELECT/INSERT/UPDATE/DELETE, role ?p d?ng nh? anon/authenticated, bi?u th?c USING ?? l?c d?ng ???c ??c/s?a/x?a v? WITH CHECK ?? ki?m tra d?ng m?i khi insert/update.

2) Theo app StaySaga: Policy StaySaga d?ng USING auth.uid() = user_id/owner_id cho ??c-s?a d? li?u c? nh?n v? WITH CHECK cho insert/update. V? d? bookings cho user ??c booking c?a m?nh, host ??c booking thu?c homestay m?nh, admin qua helper role trong migration m?i.

Câu 10. Supabase Realtime hoạt động dựa trên cơ chế nào? Khi nào nên dùng?

1) ??? ?n chu:n: Supabase Realtime d?ng WebSocket ?? g?i s? ki?n thay ??i d? li?u PostgreSQL, broadcast ho?c presence t?i client ?? subscribe. N?n d?ng khi UI c?n c?p nh?t t?c th?i nh? chat, th?ng b?o, tr?ng th?i ??n; kh?ng n?n l?m d?ng cho d? li?u ?t thay ??i.

2) Theo app StaySaga: StaySaga d?ng RealtimeSubscription ?? subscribe thay ??i bookings, reviews, homestays, profiles v? notifications. Khi c? thay ??i, UI refresh ho?c t?ng badge th?ng b?o ?? host/admin/guest th?y d? li?u m?i nhanh h?n.

Câu 11. Container khác với Virtual Machine (VM) ở điểm nào? Nêu ít nhất 3 điểm.

1) ??? ?n chu:n: Container d?ng chung kernel v?i host, nh? v? kh?i ??ng nhanh; VM c? h? ?i?u h?nh/kernel ri?ng, c? l?p m?nh h?n nh?ng n?ng h?n. Container ??ng g?i app v? dependency nh?t qu?n, c?n VM ph? h?p khi c?n c? l?p to?n h? ?i?u h?nh.

2) Theo app StaySaga: StaySaga ??ng g?i b?ng container ?? ch?y gi?ng nhau gi?a m?y dev v? VPS. So v?i VM, container nh? h?n, build/deploy nhanh h?n v? ph? h?p app Next.js + Supabase v? database n?m ngo?i ? Supabase cloud.

Câu 12. Dockerfile và docker-compose.yml có vai trò gì? Sự khác nhau giữa hai file này?

1) ??? ?n chu:n: Dockerfile m? t? c?ch build image: base image, copy file, c?i dependency, build v? command ch?y. docker-compose.yml m? t? c?ch ch?y m?t ho?c nhi?u service, port, env, volume, network. Dockerfile t?o image; Compose ?i?u ph?i container t? image ??.

2) Theo app StaySaga: Dockerfile StaySaga build Next.js standalone; docker-compose.yml ??nh ngh?a service staysaga-web, map \${APP_PORT:-3000}:3000 v? truy?n bi?n m?i tr?ng Supabase/Stripe. Dockerfile t?o image, Compose ch?y container production.

Câu 13. Multi-stage build trong Docker là gì? Tại sao nên dùng?

1) ??? ?n chu:n: Multi-stage build chia Dockerfile th?nh nhi?u stage, v? d? stage build c? compiler/dev dependencies v? stage runtime ch? copy artifact c?n ch?y. K?t qu? l? image cu?i nh? h?n, ?t b? m?t t?n c?ng h?n v? r? r?ng h?n v? build/runtime.

2) Theo app StaySaga: Dockerfile c?a StaySaga c? stage deps, builder v? runner. Runner ch? copy public, .next/standalone, .next/static v? ch?y node server.js b?ng user nextjs, gi?p image nh? v? kh?ng mang dev dependencies.

Câu 14. Các câu lệnh Docker cơ bản nào cần biết? (docker build, docker run, docker compose up/down, docker ps, docker logs)

1) ??? ?n chu:n: docker build t?o image; docker run ch?y container; docker compose up kh?i t?o service theo compose; docker compose down d?ng/x?a service; docker ps xem container; docker logs xem log; docker exec v?o container ?? debug.

2) Theo app StaySaga: Khi demo/deploy StaySaga c?n npm run build, docker compose up --build, docker compose logs -f staysaga, docker compose ps v? docker compose down. Khi l?i production, logs l? ngu?n ki?m tra ??u ti?n.

Câu 15. HTTPS và HTTP khác nhau ở điểm nào? Tại sao cần HTTPS cho production?

1) ??p ??n chu?: HTTP truy?n d? li?u d?ng kh?ng m? h?a; HTTPS l? HTTP ch?y qua TLS ?? m? h?a, x?c th?c server v? gi?m nguy c? nghe l?n/MITM. Production c?n HTTPS ?? b?o v? cookie, token, thanh to?n, form ??ng nh?p v? t?ng ?? tin c?y.

2) Theo app StaySaga: StaySaga c? ??ng nh?p, cookie session, th?ng tin ??t ph?ng v? c? th? thanh to?n, n?n production b?t bu?c HTTPS. N?u kh?ng, token/cookie v? d? li?u kh?ch c? nguy c? b? nghe l?n tr?n m?ng.

Câu 16. Reverse proxy (ví dụ Nginx) là gì? Trong mô hình deploy với Cloudflare + VPS, reverse proxy đóng vai trò gì?

1) ??p ??n chu?: Reverse proxy l? l?p nh?n request thay app, r?i chuy?n ti?p ??n app server n?i b?. V?i Cloudflare + VPS, Nginx th?ng nh?n traffic 80/443, terminate SSL ho?c d?ng origin cert, set header proxy v? chuy?n ??n Next.js ? port 3000.

2) Theo app StaySaga: Trong m? h?nh StaySaga, Cloudflare tr? domain v? VPS, Nginx tr?n VPS nh?n request v? proxy v?o container Next.js port 3000. Nginx c?ng l? ch? c?u h?nh SSL, header X-Forwarded-* v? redirect HTTP sang HTTPS.

Câu 17. Domain và DNS hoạt động như thế nào? Khi người dùng nhập example.com, quy trình DNS resolution ra sao?

1) ??p ??n chu?: Domain l? t?n d? nh? tr? ??n ??a ch? IP qua DNS. Khi nh?p domain, browser h?i resolver; resolver l?n theo root, TLD, authoritative nameserver ?? l?y record A/AAAA/CNAME, sau ?? browser k?t n?i t?i IP tr? v?.

2) Theo app StaySaga: V?i StaySaga, domain v? d? staysaga.vn s? c? DNS record tr?n Cloudflare tr? t?i IP VPS. Ng??i d?ng nh?p domain, DNS resolve ra Cloudflare/Origin, request ?i qua Cloudflare r?i ??n Nginx v? container app.

Câu 18. Cloudflare trong mô hình deploy đóng vai trò gì? Nêu ít nhất 3 chức năng.

1) ??p ??n chu?: Cloudflare c? th? l?m DNS, CDN/cache, reverse proxy edge, Universal SSL, ch?ng DDoS, WAF/rate limiting v? ?n IP origin. N? gi?p t?ng t?c, gi?m t?i VPS v? th?m l?p b?o v? tr??c khi request v?o server.

2) Theo app StaySaga: Cloudflare gi?p StaySaga qu?n l? DNS, b?t SSL mi?n ph?, cache asset t?nh, ch?ng DDoS c? b?n v? ?n IP VPS. ?i?u n?y ph? h?p ?? ?n v? gi?m ph?n v?n h?nh h? t?ng th? c?ng.

Câu 19. Sự khác biệt giữa interface và type trong TypeScript là gì? Khi nào dùng interface, khi nào dùng type?

1) ??p ??n chu?: interface ph? h?p m? t? shape object/class v? c? declaration merging; type linh ho?t h?n cho union, intersection, tuple, primitive alias v? ki?u ph?c t?p. Th?ng d?ng interface cho model object ?n ??nh, type cho union ho?c composition.

2) Theo app StaySaga: StaySaga d?ng TypeScript cho type c?a booking, homestay, profile v? props UI. Interface ph? h?p m? t? row/model object, c?n type d?ng cho union role/status nh? USER | PARTNER | ADMIN ho?c BookingStatus.

Câu 20. SSR, CSR, SSG, ISR là gì? Nêu tên đầy đủ và bản chất mỗi khái niệm.

1) ??p ??n chu?: SSR render HTML t?i m?i request; CSR render ch? y?u trong browser b?ng JavaScript; SSG render s?n l?c build; ISR cho ph?p trang t?nh ???c t?i t?o theo chu k? ho?c khi revalidate. Ch?n theo m?c ?? ??ng v? y?u c?u SEO.

2) Theo app StaySaga: StaySaga d?ng Server Component/SSR cho trang c?n d? li?u m?i nh? bookings, host/admin dashboard; trang c?ng khai c? th? t?n d?ng cache/static n?u d? li?u ?t ??i. CSR ch? d?ng cho ph?n t?ng t?c nh? filter, modal, favorite.

Câu 21. type annotation và type inference trong TypeScript khác nhau như thế nào? Ví dụ minh họa.

1) ??p ??n chu?: Type annotation l? khai b?o ki?u r? r?ng nh? let age: number. Type inference l? TypeScript t? suy ki?u t? gi? tr? nh? let age = 20. Annotation h?u ?ch ? boundary/API; inference gi?p code g?n ? bi?n c?c b?.

2) Theo app StaySaga: Trong StaySaga, n?n annotation ? boundary nh? payload Server Action, Supabase row type v? component props. Bi?n c?c b? nh? totalPrice, nights c? th? ?? TypeScript inference ?? code g?n m? v?n type-safe.

Câu 22. GitHub Copilot và Gemini CLI hoạt động dựa trên nguyên lý nào? Nêu cách sử dụng cơ bản của từng công cụ.

1) ??p ??n chu?: Copilot v? Gemini CLI d?a tr?n m? h?nh ng?n ng? l?n ph?n t?ch prompt, code context v? file li?n quan ?? sinh g?i ?. Copilot th?ng g?i ? inline trong IDE; Gemini CLI t?ng t?c qua terminal ?? h?i, s?a, t?m t?t ho?c ch?y workflow c? ki?m so?t.

2) Theo app StaySaga: StaySaga c? docs/AI_PROMPTS.md ghi c?c prompt d?ng AI ?? s?a hydration, UI homestays, mock bookings. C? th? tr?nh b?y r?ng Copilot/Gemini h? tr? sinh ? t?ng/code, c?n nh?m ki?m tra l?i b?ng build, lint v? test.

Câu 23. Các loại minh chứng sử dụng AI tool mà bạn có thể cung cấp cho giảng viên là gì?

1) ??p ?n chu?n: Minh chứng AI có thể gồm file README mô tả cách dùng AI, lịch sử chat/screenshot, prompt trong docs, commit message ghi AI-assisted, diff trước/sau, comment gì đó thể hiện AI đã xuất hiện và phần nào không thể không có.

2) Theo app StaySaga: Minh chứng sẽ nằm ở trong StaySaga gồm docs/AI_PROMPTS.md, README, commit history nếu có, các script sinh báo cáo và tài liệu QA. Khi viết xong có thể thêm file README để chứng minh AI đã được dùng để làm gì.

Câu 24. Tại sao Next.js mặc định render bằng Server Component? Lợi ích chính của việc này là gì?

1) ??p ?n chu?n: Next.js sử dụng Server Component để giảm tải JavaScript client, truy cập trực tiếp tài nguyên server, cải thiện SEO/FCP và tránh logic nặng client. Client Component chỉ dùng để render gì đó tương tác.

2) Theo app StaySaga: StaySaga là Server Component để tránh gửi dữ liệu JS về các trang danh sách/chi tiết, các Supabase bằng cookie server và không logic service/server logic. Chỉ các nút favorite, realtime, form tương tác chuyển sang client.

Câu 25. Trong Next.js, khi nào bạn cần dùng "use client" directive? Nếu không có directive này thì mặc định component hoạt động ra sao?

1) ??p ?n chu?n: Dùng "use client" khi component cần hook React client, event handler, browser API, thì vì cần browser hoặc realtime subscription. Nếu không có directive, component trong App Router mặc định là Server Component.

2) Theo app StaySaga: Trong StaySaga, "use client" dùng cho component cần useState/useEffect, router.push, toast, modal, realtime subscription hoặc click handler. Nếu quên directive các component này, Next.js sẽ báo lỗi và mặc định là Server Component.

Câu 26. Giải thích luồng xử lý một request trong Next.js App Router, từ khi user truy cập URL đến khi nhận được nội dung.

1) ??p ?n chu?n: Luồng App Router: request vào server, Next match route segment, chọn layout cha/con, có thể hiển thị loading trong khi stream, Server Component fetch dữ liệu và render RSC payload/HTML, client nhận nội dung rồi hydrate các Client Component.

2) Theo app StaySaga: Luồng StaySaga: user vào /homestays hoặc /bookings, App Router match route, layout bằng shell, Server Component tạo Supabase server client các session/cookie, query DB theo RLS, render HTML/RSC, sau các component như favorite/realtime các hydrate client.

Câu 27. So sánh Server Actions và API Routes trong Next.js. Khi nào dùng cái nào?

1) ??p ?n chu?n: Server Actions phù hợp mutation như bấm, form submit, cập nhật DB và revalidate và gọi như function và giảm boilerplate API. Route Handlers/API Routes phù hợp endpoint cần khai, webhook, thì cần xử lý thì response thì chọn hoặc cần REST contract.

2) Theo app StaySaga: StaySaga dùng Server Actions cho login/logout, booking, favorite, review, admin updates và mutation như bấm. API Route phù hợp cho webhook thành tạo endpoint bên ngoài như geocode/places và cần URL public và không có request/response.

Câu 28. Giải thích cách Supabase Auth quản lý session. Access token và refresh token hoạt động như thế nào?

1) ??p ?n chu?n: Supabase Auth phát hành access token dùng JWT và xác thực request người dùng và refresh token để lấy access token mới khi hết hạn. Trên SSR, token thì dùng các query lấy qua cookie an toàn server các session và refresh khi cần.

2) Theo app StaySaga: StaySaga dùng @supabase/ssr để createServerClient các ghi cookie session trong Server Component/Action và proxy.ts để xử lý session/báo và route. Access token dùng xác thực request, refresh token gửi duy trì dùng như bấm không bắt user login lại lần nữa.

Câu 29. Khi nào nên dùng Supabase Storage thay vì lưu file dạng base64 trong database? Ưu điểm của Storage là gì?

1) ??p ?n chu?n: Dùng Storage cho ảnh/video/file và hỗ trợ upload/download, MIME, size limit, signed URL/public URL, policy riêng và không lưu database phức tạp. Base64 trong DB làm dữ liệu nặng, query chậm và khó cache/CDN.

2) Theo app StaySaga: Ảnh homestay của StaySaga lưu trong bucket homestay-images, database chỉ lưu url/storage_path trong homestay_images. Chọn này nhờ DB, để phần query upload theo host và hiển thị ảnh cần khai/signed URL tùy policy.

C?u 30. Mô tả quy trình tạo một RLS Policy để user chỉ đọc được dữ liệu của chính mình trong Supabase.

1) ??p ?n chu?n: Quy tr?nh: b?ng c? user_id/owner_id, b?t RLS, t?o SELECT policy cho authenticated v?i USING auth.uid() = user_id, t?o INSERT/UPDATE policy c? WITH CHECK t??ng ?ng, r?i test b?ng ?t nh?t hai t?i kho?n ?? b?o ??m kh?ng ??c ch?o d? li?u.

2) Theo app StaySaga: Trong StaySaga, bookings c? user_id v? homestay_id. Policy SELECT cho user d?ng auth.uid() = user_id; host d?ng EXISTS homestays where owner_id = auth.uid(); insert c? WITH CHECK auth.uid() = user_id ?? user ch? t?o booking c?a m?nh.

C?u 31. Sự khác biệt giữa Realtime và polling trong việc cập nhật dữ liệu là gì? Ưu/nhược điểm từng cách?

1) ??p ?n chu?n: Polling l? client h?i server ??nh k? n?n ??n gi?n nh?ng t?n request v? c? ?? tr?. Realtime l? server ??y event qua WebSocket khi c? thay ??i n?n nhanh v? ti?t ki?m h?n l?c ?t event, ??i l?i c?n qu?n l? k?t n?i, quy?n subscribe v? fallback.

2) Theo app StaySaga: StaySaga d?ng Realtime cho booking/review/notification ?? admin/host th?y thay ??i nhanh. N?u d?ng polling, host dashboard ph?i g?i l?i API ??nh k? g?y t?n request; realtime ph? h?p h?n cho badge th?ng b?o v? ??n m?i.

C?u 32. Giải thích tại sao file docker-compose.yml thường dùng volume để mount source code thay vì copy vào image.

1) ??p ?n chu?n: Trong dev, volume mount source code gi?p container th?y thay ??i ngay v? h? tr? hot reload, kh?ng c?n rebuild image li?n t?c. Trong production th??ng copy code v?o image ?? image b?t bi?v n? d? rollback.

2) Theo app StaySaga: Khi dev StaySaga c? th? mount source ?? hot reload v?i npm run dev. Docker production hi?n copy standalone build v?o image ?? container ?n ??nh, ??ng v?i m?c ti?u deploy VPS.

C?u 33. Khi chạy docker compose up, Docker thực hiện những bước nào? Trình bày luồng xử lý.

1) ??p ?n chu?n: docker compose up ??c compose file, resolve env, build ho?c pull image, t?o network/volume, t?o container, mount volume, inject environment, map port, r?i start service theo dependency/healthcheck n?u c?.

2) Theo app StaySaga: docker compose up --build c?a StaySaga ??c docker-compose.yml, build Dockerfile, t?o service staysaga-web, map port 3000, inject NEXT_PUBLIC_SUPABASE_URL/KEY v? service role n?u c?, r?i ch?y server.js t? Next standalone.

C?u 34. Tại sao Multi-stage build giúp giảm kích thước Docker image? Giải thích cơ chế.

1) ??p ?n chu?n: Multi-stage gi?m image v? stage cu?i kh?ng ch?a compiler, cache build, source th?a v? dev dependencies. N? ch? copy th? m?c build/runtime c?n thi?t t? stage tr??c sang image ch?y th?t.

2) Theo app StaySaga: Image StaySaga nh? h?n v? stage runner ch? c?n Node runtime, .next/standalone, .next/static v? public. npm cache, source th?a v? dev dependencies kh?ng ?i v?o image cu?i.

C?u 35. Trong mô hình Cloudflare + VPS + Nginx, trình bày luồng request từ browser đến ứng dụng Next.js.

1) ??p ?n chu?n: Browser truy c?p domain, DNS/Cloudflare tr? edge ho?c origin theo c?u h?nh, Cloudflare nh?n TLS v? proxy t?i VPS, Nginx tr?n VPS nh?n 80/443 r?i proxy_pass t?i Next.js container/app port 3000, response ?i ng??c l?i browser.

2) Theo app StaySaga: Lu?ng production StaySaga: browser -> domain DNS Cloudflare -> Cloudflare SSL/proxy -> VPS -> Nginx -> container staysaga-web port 3000 -> Next.js App Router -> Supabase cloud n?u c?n d? li?u -> tr? response.

C?u 36. SSL Certificate (chứng chỉ SSL) hoạt động như thế nào? Giải thích cơ chế mã hoá trong HTTPS handshake.

1) ??p ?n chu?n: Trong TLS handshake, client v? server th??ng l??ng phi?n b?n/cipher, server g?i certificate c? public key v? ch? k? CA, client x?c th?c certificate, hai b?n sinh kh?a phi?n ??i x?ng r?i d?ng kh?a ?? ?? m? h?a d? li?u HTTP.

2) Theo app StaySaga: V?i StaySaga, TLS b?o v? cookie Supabase Auth v? th?ng tin booking. Cloudflare/Nginx tr?nh ch?ng ch? cho browser; sau handshake, d? li?u nh? login, checkout, profile ???c m? h?a b?ng session key.

C?u 37. Cloudflare Universal SSL hoạt động như thế nào? Ưu điểm so với tự cài SSL trên VPS?

1) ??p ?n chu?n: Cloudflare Universal SSL t? c?p v? gia h?n ch?ng ch? t?i edge cho domain d?ng Cloudflare. ?u ?i?m l? nhanh, mi?n ph?, ?t thao t?c, t?ch h?p CDN/WAF; v?n n?n c?u h?nh Full/Strict v?i certificate h?p l? ? origin ?? m? h?a to?n tuy?n.

2) Theo app StaySaga: StaySaga c? th? d?ng Cloudflare Universal SSL cho edge v? Origin Certificate/Let's Encrypt ? VPS. C?u h?nh n?n l? Full Strict ?? m? h?a c? ?o?n Cloudflare t?i VPS, kh?ng ch? browser t?i Cloudflare.

C?u 38. Giải thích khái niệm generics trong TypeScript. Cho ví dụ một hàm generic đơn giản.

1) ??p ?n chu?n: Generics cho ph?p vi?t h?m/ki?u d?ng ???c v?i nhi?u ki?u d? li?u nh?ng v?n gi? type safety. V? d? function identity<T>(value: T): T { return value; } tr? v? ??ng ki?u ?? truy?n v?o.

2) Theo app StaySaga: StaySaga c? th? d?ng generic cho helper tr? v? d? li?u Supabase ho?c form state, v? d? Result<T> = { data?: T; error?: string }. Nh? v?y action booking, review, profile c? ki?u tr? v? nh?t qu?n.

C?u 39. So sánh SSR và SSG. Khi nào nên dùng SSG, khi nào nên dùng SSR cho một trang web?

1) ??p ?n chu?n: SSG render l?c build n?n r?t nhanh v? h?p n?i dung t?nh nh? docs/blog/landing. SSR render theo request n?n h?p d? li?u c? nh?n h?a ho?c thay ??i th??ng xuy?n nh? dashboard, profile, ??n h?ng.

2) Theo app StaySaga: Trang landing ho?c danh s?ch th?nh ph? c?a StaySaga c? th? SSG/ISR n?u d? li?u ?t ??i; dashboard /host, /admin, /bookings n?n SSR v? ph? thu?c session, role v? d? li?u booking m?i nh?t.

C?u 40. Tại sao nên dùng TypeScript thay vì JavaScript thuần trong dự án Next.js? Liệt kê các lợi ích chính.

1) ??p ?n chu?n: TypeScript ph?t hi?n l?i ki?u tr??c runtime, h? tr? autocomplete, refactor an to?n, t?i li?u h?a contract qua type/interface v? gi?m bug khi codebase l?n. V?i Next.js, n? gi?p ki?m so?t props, API payload v? d? li?u DB t?t h?n.

2) Theo app StaySaga: StaySaga d?ng TypeScript ?? ki?m so?t props, role/status, payload booking v? d? li?u tr? v? t? Supabase. ?i?u n?y gi?m l?i khi refactor route/action v? gi?p IDE c?nh b?o tr??c khi build.

C?u 41. Trong dự án của bạn, bạn đã thiết kế database schema như thế nào? Vì sao bạn chọn cấu trúc đó thay vì cách khác?

1) ??p ?n chu?n: Schema n?n chu?n h?a theo th?c th?, quan h? 1-n/n-n r? r?ng, kh?a ngo?i, index theo truy v?n v? RLS theo owner/user. Ch?n relational khi c?n query/ch?ng tr?ng; denormalize c? ki?m so?t cho th?ng k? ho?c d? li?u ???c nhi?u.

2) Theo app StaySaga: Schema StaySaga t?ch profiles, homestays, homestay_images, amenities, homestay_amenities, bookings, favorites, reviews, notifications. T?ch b?ng gi?p query r?, tr?nh l?p d? li?u, ??t kh?a ngo?i/RLS v? m? r?ng host/admin d? h?n.

C?u 42. Khi Supabase không hỗ trợ một tính năng nhất định (ví dụ: complex job queue), bạn sẽ xử lý như thế nào?

1) ??p ?n chu?n: N?u Supabase thi?u job queue ph?c t?p, c? th? d?ng Edge Functions, cron, Trigger.dev/Inngest, queue ngo?i nh? Redis/BullMQ ho?c b?ng jobs ??n gi?n c? worker polling. Ch?n theo ?? tin c?y, retry, volume v? chi ph?.

2) Theo app StaySaga: N?u StaySaga c?n job queue nh? g?i email nh?c check-in, c? th? d?ng Supabase Edge Functions + cron ho?c Trigger.dev/Inngest. V?i ?? ?n nh?, b?ng notifications/jobs + worker ??nh k? l? ??; khi scale m?i d?ng queue chuy?n d?ng.

C?u 43. Bạn đã sử dụng Server Components và Client Components như thế nào trong dự án? Có gặp khó khăn gì khi phân chia không?

1) ??p ?n chu?n: Nguy?n t?c l? Server Component cho ??c d? li?u, auth gate, SEO/static content; Client Component cho form state, modal, toast, realtime v? event. Kh? kh?n th??ng l? props ph?i serializable, ranh gi?i "use client" v? tr?nh g?i server-only code t? client.

2) Theo app StaySaga: StaySaga d?ng Server Component cho page ??c Supabase v? auth gate; Client Component cho FavoriteButton, wizard host, modal ??i gi?/ng?y, RealtimeSubscription. Kh? kh?n ch?nh l? kh?ng truy?n ???c object kh?ng serializable qua ranh gi?i server-client v? ph?i t?ch action ??ng ch?.

C?u 44. Thiết kế chiến lược phân quyền (authorization) cho ứng dụng của bạn. Ai được làm gì, kiểm tra ở đâu (client? server? database?)

1) ??p ?n chu?n: Authorization n?n c? nhi?u l?p: UI ch? hi?n th? h?nh ??ng h?p l?, server ki?m tra session/role tr??c mutation, database RLS l? l?p ch?n cu?i c?ng. Kh?ng tin client v? ng??i d?ng c? th? t? g?i request ho?c s?a payload.

2) Theo app StaySaga: StaySaga ph?n quy?n Guest/USER/PARTNER/ADMIN. Guest xem public; USER ??t ph?ng, favorite, review; PARTNER qu?n l? homestay/booking c?a m?nh; ADMIN qu?n l? h? th?ng. Ki?m tra ? UI, server actions/pages v? RLS database ?? kh?ng ph? thu?c client.

C?u 45. Nếu ứng dụng Next.js của bạn bị crash khi chạy trong Docker container trên VPS, bạn sẽ debug bằng cách nào? Trình bày các bước.

1) ??p ?n chu?n: Debug container crash: xem docker compose ps/logs, ki?m tra exit code, env, secret, port mapping, build artifact, node version, network/DNS, t?i nguy?n CPU/RAM/disk; th? curl n?i b? container/VPS v? ch?y command start tr?c ti?p n?u c?n.

2) Theo app StaySaga: N?u StaySaga crash trong Docker, ki?m tra docker compose logs -f staysaga, env Supabase, NEXT_PUBLIC_SITE_URL, port APP_PORT, output .next/standalone, npm run build v? th? curl localhost:3000 tr?n VPS. C?c l?i schema Supabase c?ng c?n ki?m tra log server action.

Câu 46. Khi deploy lên VPS, bạn cần cấu hình Nginx như thế nào để proxy request đến Next.js app? Cho ví dụ cấu hình cơ bản.

1) ??? ?n chu?: Nginx c?n server_name domain, listen 80/443, SSL certificate, location / proxy_pass t?i http://127.0.0.1:3000, set Host, X-Real-IP, X-Forwarded-For, X-Forwarded-Proto v? websocket headers n?u app d?ng realtime/socket.

2) Theo app StaySaga: Nginx cho StaySaga s? proxy / v? http://127.0.0.1:3000 ho?c t?n service Docker, set Host/X-Forwarded-Proto/X-Real-IP, redirect HTTP sang HTTPS v? t?ng client_max_body_size n?u upload ?nh homestay l?n.

Câu 47. Giả sử bạn cần deploy 2 ứng dụng (frontend Next.js + backend NestJS) trên cùng một VPS. Thiết kế cấu hình Docker Compose và Nginx.

1) ??? ?n chu?: Thi?t k? g?m docker-compose c? service nextjs v? nestjs c?ng network n?i b?. Nginx route /api/ t?i nestjs:3001 v? / t?i nextjs:3000, th?m healthcheck, restart policy, env t?ch ri?ng v? ch? expose Nginx ra public.

2) Theo app StaySaga: N?u th?m backend NestJS cho StaySaga, Compose c? service web:3000 v? api:3001 c?ng network. Nginx route /api/ t?i api:3001, c?n / t?i web:3000; c? hai d?ng env ri?ng v? ch? Nginx expose 80/443 ra ngo?i.

Câu 48. Bạn cần hiển thị danh sách bài viết cho user chưa đăng nhập (anonymous). Nêu cách cấu hình RLS Policy cho trường hợp này.

1) ??? ?n chu?: V?i d? li?u public cho anonymous, b?t RLS v? t?o SELECT policy cho role anon ho?c public condition nh? published = true. C?c thao t?c ghi v?n gi?i h?n authenticated/admin ?? tr?nh ng??i l? s?a d? li?u.

2) Theo app StaySaga: Trong StaySaga, danh s?ch homestay public cho guest n?n c? policy SELECT cho anon/authenticated v?i ?i?u ki?n is_active = true ho?c status = approved. D? li?u draft/?n c?a host kh?ng ???c tr? cho anon.

Câu 49. Nếu bạn cần upload file lớn (video, 50MB) lên Supabase Storage, bạn cần lưu ý những gì về cấu hình và RLS?

1) ??? ?n chu?: Upload file l?n c?n ki?m tra gi?i h?n bucket/project, MIME, k?ch th??c, timeout, resumable upload n?u c?, quy?n insert/select trong storage.objects, ???ng d?n theo user_id v? d?ng signed URL n?u file private.

2) Theo app StaySaga: V?i ?nh/video l?n cho homestay, StaySaga c?n c?u h?nh bucket homestay-images v? size/MIME, d?ng ???ng d?n owner_id/property_id, policy cho host upload property c?a m?nh, signed URL n?u private v? progress/resumable upload n?u file l?n.

Câu 50. Thiết kế hệ thống notification real-time (thông báo khi có tin nhắn mới) sử dụng Supabase Realtime. Trình bày flow từ client đến server.

1) ??? ?n chu?: Flow notification: user/action t?o record notification ho?c message trong DB, RLS ??m b?o ch? ng??i nh?n ???c ???c, client ng??i nh?n subscribe channel/filter theo user_id, Supabase Realtime g?i INSERT event, UI t?ng badge/toast v? c? th? mark read.

2) Theo app StaySaga: Notification StaySaga: khi c? booking/message/review m?i, server insert v?o notifications v?i user_id ng??i nh?n; client subscribe notifications:user_id qua Supabase Realtime; khi nh?n INSERT th? t?ng badge, hi?n toast v? refresh danh s?ch.

Câu 51. Trong quá trình phát triển, bạn đã sử dụng AI tool như thế nào? Chọn một tính năng cụ thể và mô tả prompt bạn dùng để tạo code đó.

1) ??? ?n chu?: N?n d?ng AI ?? sinh skeleton, gi?i th?ch l?i, t?o test case ho?c refactor nh?. Prompt t?t c?n c? framework, file li?n quan, y?u c?u ??u ra, constraint b?o m?t v? ti?u ch? ki?m th?; sau ?? developer review, ch?nh code v? ch?y test.

2) Theo app StaySaga: M?t v? d? StaySaga: d?ng AI ?? thi?t k? UI /homestays theo phong c?ch Booking/Agoda ho?c s?a hydration App Router. Prompt n?u Next.js App Router, TypeScript, Tailwind, y?u c?u gi? Server/Client boundary; sau ?? nh?m ch?nh l?i theo codebase v? ch?y build/test.

Câu 52. Nêu ít nhất 3 trường hợp mà AI tool không nên dùng hoặc cần cẩn thận khi dùng trong dự án.

1) ??? ?n chu?: C?n c?n th?n khi d?ng AI cho b?o m?t, thanh to?n, ph?n quy?n/RLS, migration ph? d? li?u, logic nghi?p v? ph?c t?p, code kh?ng hi?u ho?c d? li?u b? m?t. AI c? th? b?a API, b? edge case ho?c sinh pattern kh?ng an to?n.

2) Theo app StaySaga: Trong StaySaga c?n c?n th?n khi AI vi?t RLS, service role, thanh to?n, migration database v? logic t?nh ti?n/ch?ng double booking. Nh?ng ph?n n?y ph?i ???c docs, review th? c?ng v? test b?ng nhi?u role tr??c khi demo.

C?u 53. Bạn đã quản lý Git commit của mình như thế nào? Tại sao dùng Conventional Commits? Cho ví dụ commit message.

1) ??p ?n chu?n: Qu?n l? Git n?n commit nh? theo t?nh n?ng/s?a l?i, message r? r?ng. Conventional Commits nh? feat: add booking checkout, fix: prevent double booking, docs: update deployment guide gi?p ??c l?ch s?, review v? t?o changelog d? h?n.

2) Theo app StaySaga: StaySaga n?n commit theo nh?m nh? nh? feat: add booking checkout, fix: correct booking RLS, docs: add deployment checklist. Conventional Commits gi?p gi?ng vi?n/nh?m th?y l?ch s? r? v? d? truy v?t l?i.

C?u 54. Đánh giá 3 điểm mạnh và 3 điểm hạn chế của kiến trúc dự án bạn đã chọn. Nếu làm lại, bạn sẽ thay đổi gì?

1) ??p ?n chu?n: ??nh gi? ki?n tr?c d?a tr?n maintainability, security, deployability, performance v? ?? ph? h?p y?u c?u. ?i?m m?nh th??ng l? full-stack g?n, type-safe, RLS m?nh; h?n ch? c? th? l? ph? thu?c BaaS, realtime/role ph?c t?p v? c?n ki?m th? migration k?.

2) Theo app StaySaga: ?i?m m?nh StaySaga: Next.js + Supabase g?n, RLS b?o m?t d? li?u, Docker standalone d? deploy. H?n ch?: ph? thu?c schema Supabase cloud, role/RLS ph?c t?p, m?t s? lu?ng host/admin c?n test th?m. N?u l?m l?i n?n ch?t migration/seed/test s?m h?n.

C?u 55. Trong 6 tiêu chí bắt buộc (Next.js, Supabase, Docker, VPS, GitHub, AI), tiêu chí nào bạn thấy khó nhất? Vì sao? Bạn đã vượt qua như thế nào?

1) ??p ?n chu?n: Ti?u ch? kh? nh?t th??ng l? RLS ho?c VPS/Docker v? l?i n?m gi?a code, config v? h? t?ng. C?ch v??t qua l? ??c docs ch?nh th?c, chia nh? test, ghi checklist, ki?m tra log v? t?o t?i li?u tri?n khai l?p l?i ???c.

2) Theo app StaySaga: V?i StaySaga, kh? nh?t l? Supabase RLS/schema cloud v? Docker/VPS v? l?i c? th? kh?ng n?m trong UI. Nh?m v??t qua b?ng ??c log, vi?t t?i li?u QA, ki?m th? route theo role v? ghi l?i bug nh? enum role, constraint homestays, lint.

C?u 56. Nếu bạn phải thêm một tính năng mới (ví dụ: thanh toán PayPal) vào dự án hiện tại, quy trình thực hiện của bạn sẽ như thế nào?

1) ??p ?n chu?n: Quy tr?nh th?m PayPal: ph?n t?ch use case, thi?t k? b?ng payments/transactions, c?p nh?t RLS, t?o API route webhook v? Server Action t?o order, t?ch h?p SDK, validate callback server-side, c?p nh?t UI, vi?t test, c?p nh?t docs v? deploy t?ng b??c.

2) Theo app StaySaga: Th?m PayPal v?o StaySaga: t?o b?ng payments li?n k?t bookings, Server Action t?o order, API Route webhook PayPal x?c th?c server-side, c?p nh?t booking/payment status, RLS ch? user/host/admin xem ??ng quy?n, UI checkout th?m PayPal, test sandbox r?i deploy.

C?u 57. Hạn chế/thách thức lớn nhất bạn gặp phải trong quá trình phát triển đồ án là gì? Bạn đã giải quyết bằng cách nào?

1) ??p ?n chu?n: Th?ch th?c n?n tr? l?i b?ng v?n ?? c? th?, v? d? ph?n quy?n/RLS, schema cloud l?ch migration ho?c Docker deploy l?i env. C?ch gi?i quy?t l? ??c log, so s?nh schema, t?o migration/fix, test l?i lu?ng ch?nh v? r?t kinh nghi?m v? ki?m so?t m?i tr??ng.

2) Theo app StaySaga: Th?ch th?c l?n c?a StaySaga l? ??ng b? codebase v?i database cloud v? ph?n quy?n host/admin. Nh?m x? l? b?ng migration, RLS policy, test route theo role, t?i li?u qa-full-test-report v? workaround demo b?ng t?i kho?n ADMIN khi enum PARTNER/HOST ch?a ??ng b?.

Ngu?n Tham Kh?o Ch?nh

? Next.js Docs: Server/Client Components, App Router, Server Actions, data fetching - <https://nextjs.org/docs>
? Supabase Docs: Auth, Row Level Security, Storage access control, Realtime - <https://supabase.com/docs>
? Docker Docs: Docker Compose v? multi-stage builds - <https://docs.docker.com>
? Cloudflare Docs: DNS, proxy, Universal SSL - <https://developers.cloudflare.com>
? MDN Web Docs: HTTP, HTTPS/TLS - <https://developer.mozilla.org>
? TypeScript Handbook: Everyday Types, Generics - <https://www.typescriptlang.org/docs>
? GitHub Docs: GitHub Copilot - <https://docs.github.com/copilot>
? Gemini CLI Docs - <https://google-gemini.github.io/gemini-cli/docs/>
? N?i b? StaySaga: README.md, supabase/migrations/202605150001_init_staysaga.sql, supabase/migrations/202605170001_roles_rls_user_partner_admin.sql, docs/qa-full-test-report.md, docs/AI_PROMPTS.md.